Student Management System

MongoDB, Database Operations and Cloud Deployment

Course detail	Information
Course title	Full-Stack Web Development
Unit	Unit V: MongoDB and Deploying Applications
Project	CampusPulse : Student Records Management System
Student	Gedela Kavya
Roll number	A24126552082
Academic year	2026–2027

PART A. Comprehensive Question

Case Study: CampusPulse Student Records Management System

An educational institution needs a reliable way to maintain student records. Paper registers and disconnected spreadsheets make it difficult to search for students, update details, prevent duplicate records and confirm whether information is current. The institution wants a web-based Student Records Management System that can be accessed through a browser.

Design and develop a database-driven student management application using React, Node.js, Express.js and MongoDB Atlas. Deploy the backend and frontend on suitable cloud platforms and make the application accessible through a web address.

The application must support the following requirements:

- Store student details such as full name, email, course, academic year and status.
- Create and manage a MongoDB database and student collection using an appropriate document structure.
- Perform CRUD operations: add a student, display students, modify student details and delete a student.
- Validate required fields, email addresses, academic years and status values.
- Prevent duplicate email addresses.
- Provide search by student name, email or course.
- Display total students, active records and courses on the dashboard.
- Provide a health endpoint that reports whether MongoDB Atlas or the demo memory store is active.
- Deploy the backend, frontend and database using cloud platforms.
- Test the application and confirm that records entered through the interface can be retrieved.

PART B. Worked Solution

1. Analyse the requirements

Requirement	Solution
Student identity	MongoDB's automatically generated `_id` identifies each record.
Student details	Name, email, course, year and status are stored in each document.
Data storage	MongoDB Atlas stores persistent records when `MONGODB_URI` is configured.
Application operations	React sends HTTP requests to Express API routes; Express reads and writes student documents.
Validation	The backend validates required fields, email format, academic year and status.
Search	The dashboard filters records by name, email or course.
Demonstration mode	Without `MONGODB_URI`, the API uses a clearly reported in-memory demo store.
Deployment	Render can host the combined Node.js application; Vercel can host the static frontend.

2. Design the MongoDB document

A student is represented as one document in the students collection:

```
{
  "name": "Aarav Sharma",
  "email": "aarav@example.com",
  "course": "Computer Science",
  "year": 3,
  "status": "Active"
}
```

Field	Suggested data type	Purpose
`name`	String	Student's full name
`email`	String	Student's unique contact email
`course`	String	Academic course
`year`	Number	Academic year from 1 to 4
`status`	String	`Active` or `Inactive`
`createdAt`	Date	Automatically generated creation time
`updatedAt`	Date	Automatically generated update time

The document also receives an automatically generated _id field. The application uses this identifier while editing and deleting a student.

3. Create the database and collection

In MongoDB Shell, the following commands can be used to create the database and sample records:

```
// MongoDB Shell: create the database and student collection use campuspulse_db


db.createCollection("students")


// Insert demonstration records db.students.insertMany([
  {     name: "Aarav Sharma",
email: "aarav@example.com",
course: "Computer Science",     year:
3,
    status: "Active"
  },
{
    name: "Diya Reddy",     email:
"diya@example.com",     course:
"Artificial Intelligence",     year:
2,     status: "Active"



  },
{
    name: "Vikram Singh",     email:
"vikram@example.com",     course: "Information
Technology",     year:
4,
    status: "Inactive"
  } ])
```

The sample values are illustrative. The application can also start with its built-in demo records when no MongoDB connection string is configured.

4. Perform MongoDB CRUD operations

The following queries are grouped by operation so that each database action can be tested independently.

Create a student

```
db.students.insertOne({   name:
"Meera Iyer",   email:
"meera@example.com",   course:
"Electronics",   year: 1,   status:
"Active"
})
```

Read students

```
db.students.find({ course: "Computer Science" })
```

Update a student

```
db.students.updateOne(
  { email: "aarav@example.com" },
  { $set: { year: 4 } }
)
```

Delete a student

```
db.students.deleteOne({ email: "meera@example.com" })
```

Useful additional queries:

```
// Display all students db.students.find({})

// Find active students
db.students.find({ status: "Active" })

// Find final-year students db.students.find({ year: 4 })

// Sort students alphabetically db.students.find({}).sort({ name: 1 })

// Count all students db.students.countDocuments({})
```

5. Create an index for student lookup

Email addresses are used to identify duplicate records. In a production MongoDB database, a unique index should be created:

```
db.students.createIndex({ email: 1 }, { unique: true })
db.students.createIndex({ course: 1 }) db.students.getIndexes()
```

The unique email index prevents two documents from using the same email address. The course index supports course-based lookup. Indexes should be selected according to the application's actual queries.

6. Develop the Express.js backend

The backend is implemented in backend/server.js. It creates an Express application, serves the sibling frontend folder, defines a Mongoose Student model and exposes REST endpoints.

Code listing 1 — backend/server.js: Mongoose student schema

```
const studentSchema = new mongoose.Schema(
  {
    name: { type: String, required: true, trim: true },
    email: { type: String, required: true, trim: true, lowercase: true },
course: { type: String, required: true, trim: true },    year: { type: Number,
required: true, min: 1, max: 4 },
    status: { type: String, enum: ["Active", "Inactive"], default: "Active" }
},
  { timestamps: true }
);
```

Code listing 2 — backend/server.js: validation rules

```
function validateStudent(body) {   const { name, email, course, year,
status = "Active" } = body;


  if (!name || !email || !course || year === undefined || year === "") {     return
"Name, email, course and year are required.";
  }   if
(!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) {     return
"Please provide a valid email address.";
  }   if (![1, 2, 3,
4].includes(Number(year))) {     return "Year must
be between 1 and 4.";
  }   if (!["Active",
"Inactive"].includes(status)) {     return
"Status must be Active or Inactive.";
  }
  return null;
}
```

Before a record is stored, validateStudent checks that:

- Name, email, course and year are present.
- The email matches a valid email pattern.
- The academic year is between 1 and 4.
- The status is either `Active` or `Inactive`.

API endpoint summary:

Method	**Endpoint**	**Action**
GET	`/api/health`	Check API and active database mode
GET	`/api/students`	Retrieve all student records
POST	`/api/students`	Create a student
PUT	`/api/students/:id`	Update a student by `_id`
DELETE	`/api/students/:id`	Delete a student by `_id`

The application returns 400 Bad Request for validation errors, 404 Not Found for missing records, 409 Conflict for duplicate email addresses and 500 Internal Server Error for unexpected failures.

7. Develop the React frontend

The frontend is implemented in frontend/app.jsx and loaded by frontend/index.html. It uses React state and browser fetch requests to communicate with the Express API.

Code listing 3 — frontend/app.jsx: loading records

```
async function loadStudents() {   setLoading(true);
try {
    const response = await fetch("/api/students");
    if (!response.ok) throw new Error("Unable to load students.");     setStudents(await
response.json());
  } catch (error) {     setMessage({ type: "error",
text: error.message });
  } finally {
    setLoading(false);
  } }
```

Code listing 4 — frontend/app.jsx: create or update

```
const method = editingId ? "PUT" : "POST"; const url
= editingId
  ? `/api/students/${editingId}`
  : "/api/students";


const response = await fetch(url, {   method,
  headers: { "Content-Type": "application/json" },   body:
JSON.stringify(form) });
```

The dashboard provides:

- An add/edit student form.
- Course, year and status selection controls. • Required-field and email input validation.
- A searchable student directory.
- Edit and delete actions.
- Total student, active record and course statistics.
- Success and error messages after API operations.

The main workflow is:

```
Dashboard loads
|
v
GET /api/students
|
v
User submits the form
      |
      +--> POST /api/students  (new record)
|
      +--> PUT /api/students/:id  (edited record)
      |
v
Reload and display the updated directory
```

The application is branded CampusPulse in the dashboard heading, browser title, footer and server startup message.

8. Run and test the application locally

From the backend directory:

Code listing 5 — PowerShell: install and start

```
cd
"CASE_STUDY\StudentManagementSystem
\backend" npm install npm start
```

Open http://localhost:5000. The application runs with the demo memory store if MONGODB_URI is not configured.

Code listing 6 — PowerShell: run automated tests

```
npm test
```

Code listing 7 — PowerShell: development mode

```
npm run dev
```

Code listing 8 — PowerShell: API checks

```
curl http://localhost:5000/api/health curl http://localhost:5000/api/students
```

Expected health response when MongoDB is not configured:

```
{
 "status": "ok",
 "database": "Demo memory store"
}
```

9. Test cases

Test case	Action	Expected result
TC1: Create	Submit a complete student form	New student is stored and displayed
TC2: Read	Open or reload the dashboard	Student records are retrieved
TC3: Update	Edit a student's course or year	Updated value is displayed after reload
TC4: Delete	Delete a student after confirmation	Student is removed from the directory
TC5: Validation	Submit missing or invalid values	API returns a validation error
TC6: Duplicate	Submit an email already in use	API returns a conflict error
TC7: Search	Search by name, email or course	Matching records are shown
TC8: Health	Open `/api/health`	Service reports `ok` and its database mode

Current automated test results:

Test	Result
Accepts a complete student record	Pass
Rejects missing required fields	Pass
Rejects invalid email and year	Pass

10. Deploy the application using cloud platforms

Deployment architecture:

Layer	Platform	Responsibility
Frontend	Vercel or Express static hosting	Hosts the CampusPulse dashboard
Backend	Render or the Node.js server	Runs the Express REST API
Database	MongoDB Atlas	Stores student documents

Deployment steps:

1. Create a MongoDB Atlas cluster and database user.
2. Configure network access according to the deployment requirements.
3. Push the project source to a Git repository without committing `.env` files.
4. On render , create a web service with root directory
5. `CASE_STUDY/StudentManagementSystem/backend`.
6. Use `npm install` as the build command and `npm start` as the start command.
7. Add `MONGODB_URI` as a private environment variable.
8. Test the deployed `/api/health` endpoint.
9. For a separate frontend deployment, import the `frontend` folder into Vercel.
10. Configure the frontend API URLs to point to the deployed backend.
11. Test add, read, edit, delete and search operations using the public application.

Do not place MongoDB usernames, passwords or connection strings in frontend code or source control. Use synthetic student data for classroom demonstrations.

11. Expected final outcome

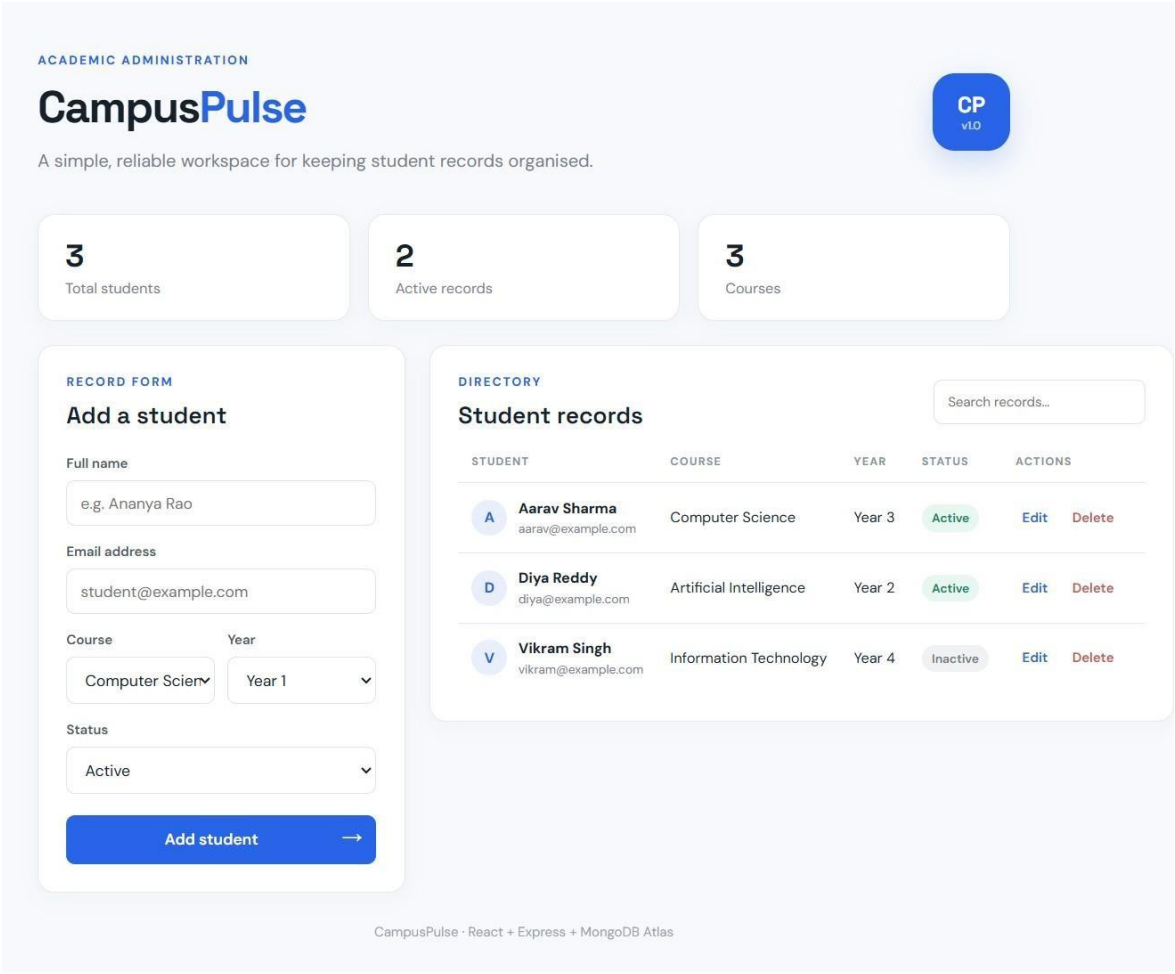
- A working CampusPulse student dashboard.
- A backend API that responds successfully to `/api/health`.
- A MongoDB Atlas database containing student documents when configured.
- Working add, view, edit and delete operations.
- Search by name, email and course.
- Validation and duplicate-email protection.
- Automated tests showing passing validation behavior.
- Evidence that the frontend communicates with the backend.

Students should capture their own screenshots during execution. Suggested evidence includes the dashboard, successful add operation, edit and refresh, search results, delete result, validation error, health endpoint and passing test output.

PART C. Output Screenshots (Test Case Results)

Screen 1: CampusPulse dashboard

Shows the CampusPulse heading, student statistics, record form and student directory.



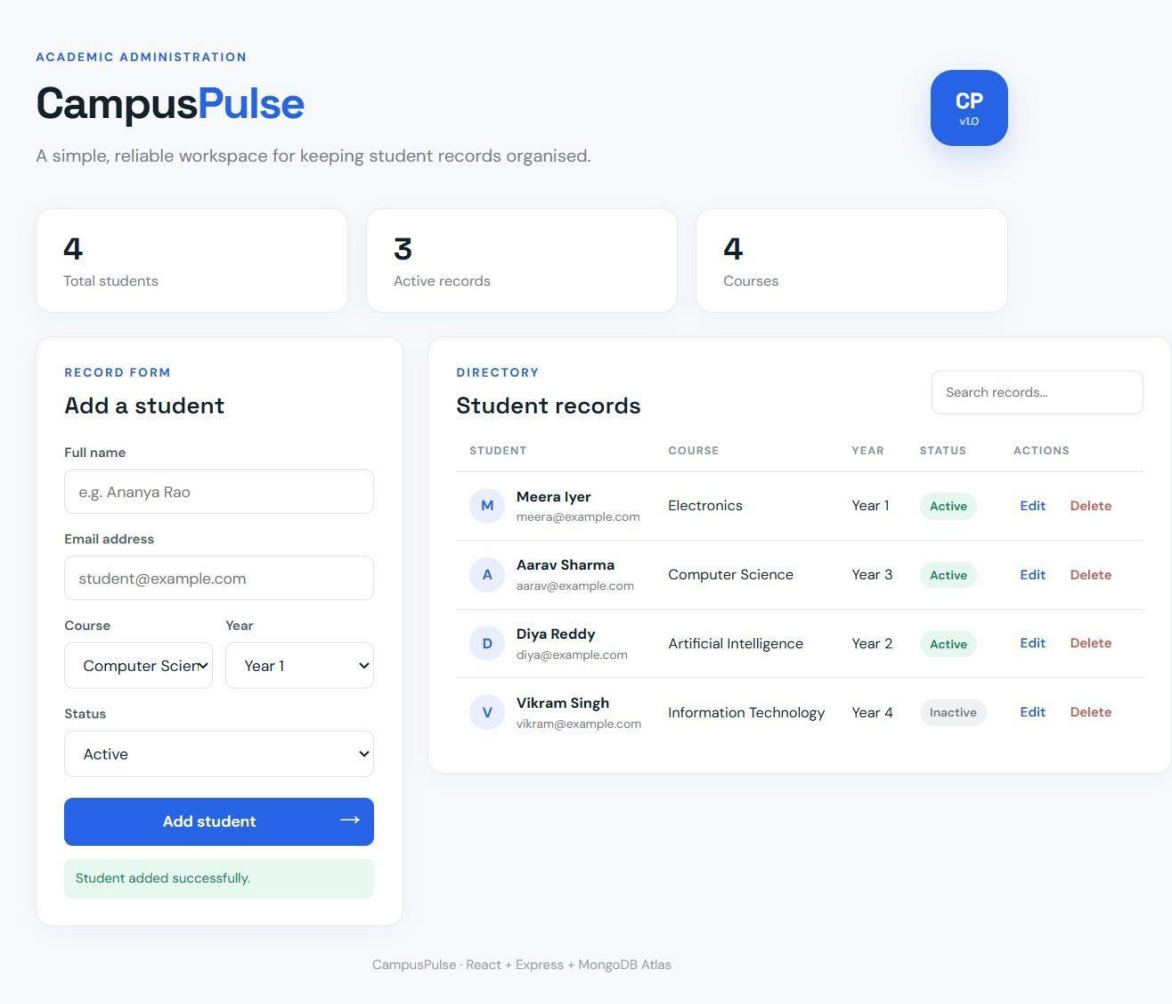
CampusPulse dashboard

Figure 1. CampusPulse dashboard running locally with the demo memory store.

Screen 2: Add student success

The form was completed with the following demonstration values:

```
Name: Meera Iyer
Email: meera@example.com
Course: Electronics
Year: Year 1
Status: Active
```

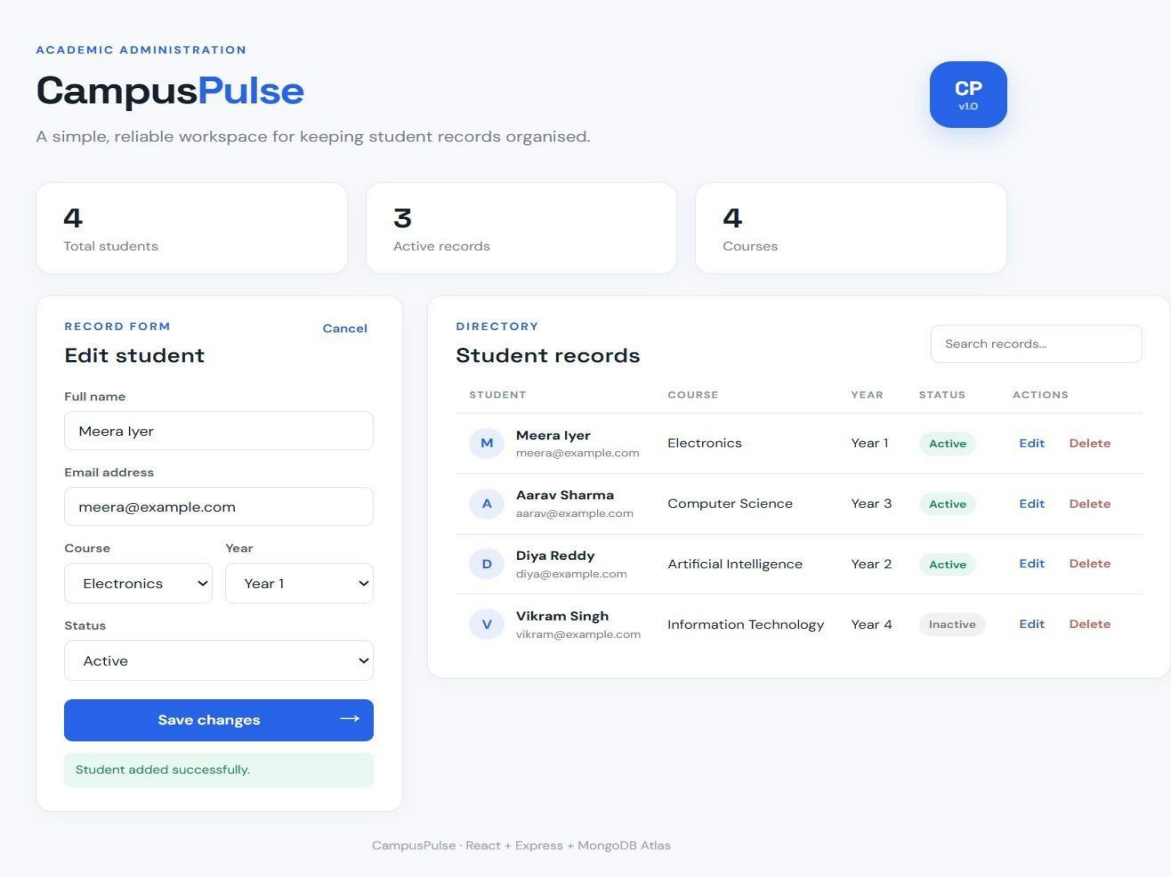


CampusPulse add success

Figure 2. CampusPulse showing a successful student creation message and the new record in the directory.

Screen 3: Edit student form

The existing student record was opened with the Edit action. The form was populated with the stored values before making an update.

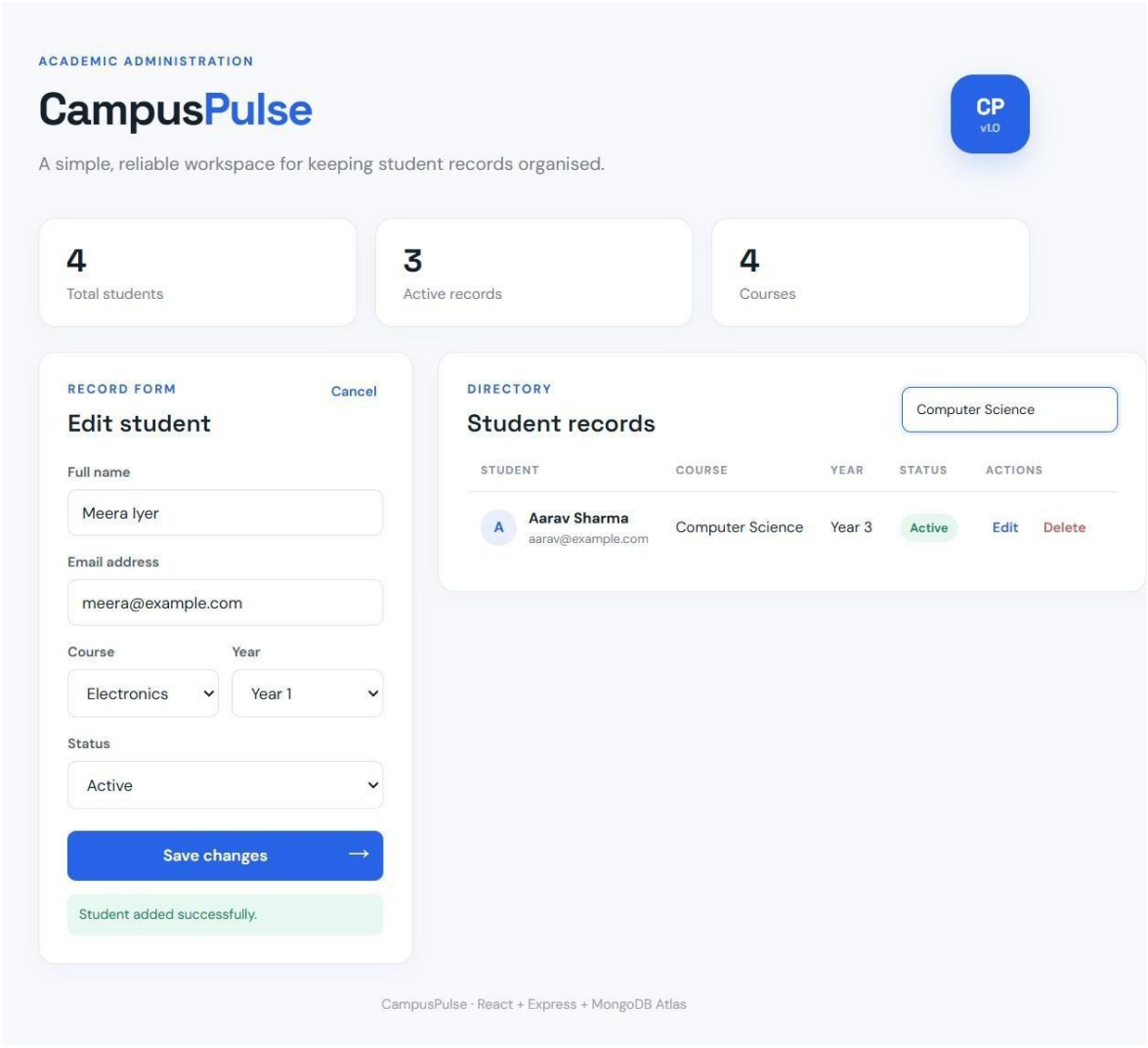


CampusPulse edit form

Figure 3. Edit student form populated with the selected student's existing details.

Screen 4: Search results

The search box was used with the query Computer Science. The directory displayed only the matching record.



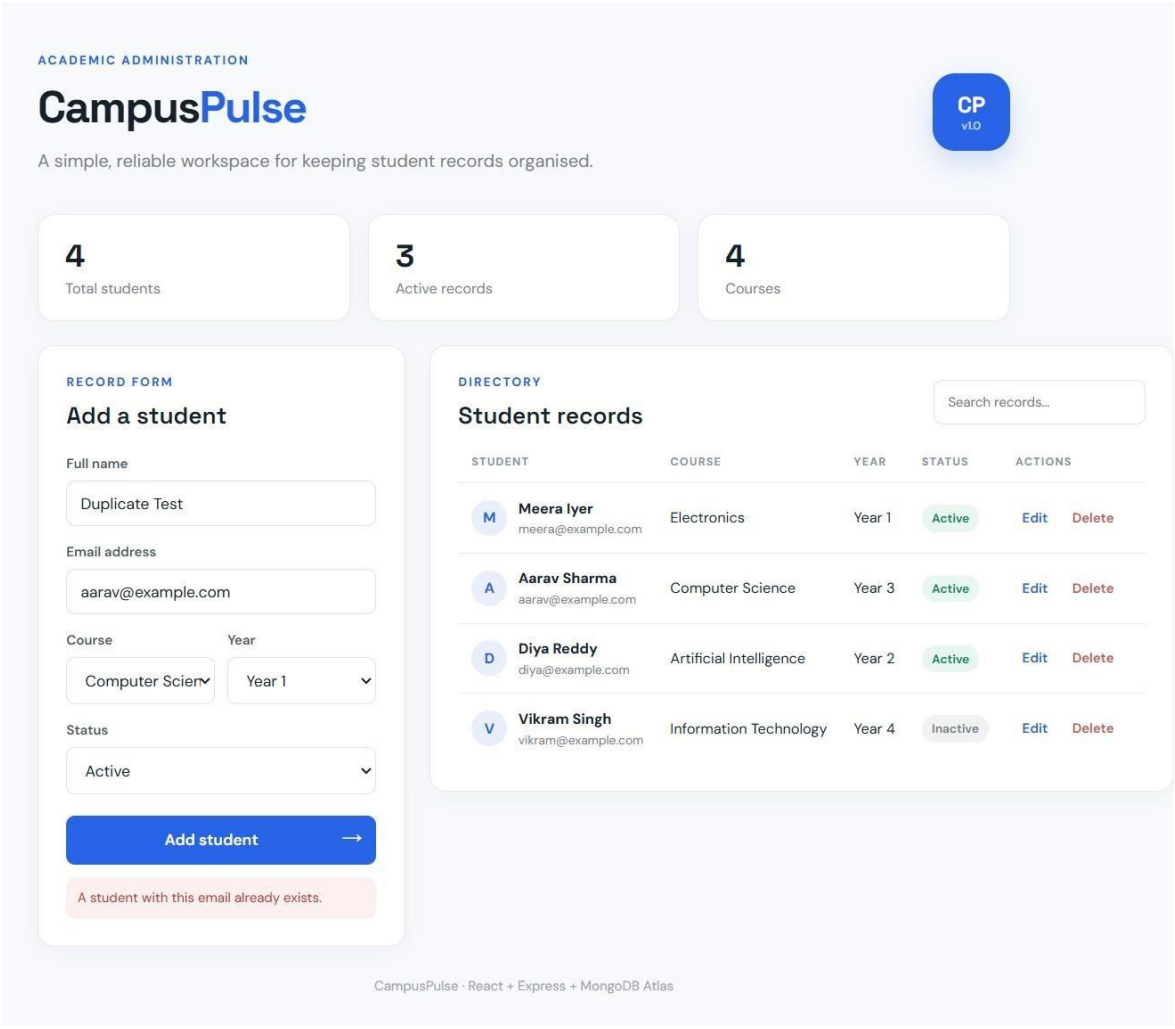
CampusPulse search results Figure 4.

CampusPulse directory filtered by course using the search field.

Screen 5: Validation and duplicate-email result

The form was submitted with an email address that already exists. The API rejected the request and displayed the following message:

> A student with this email already exists.

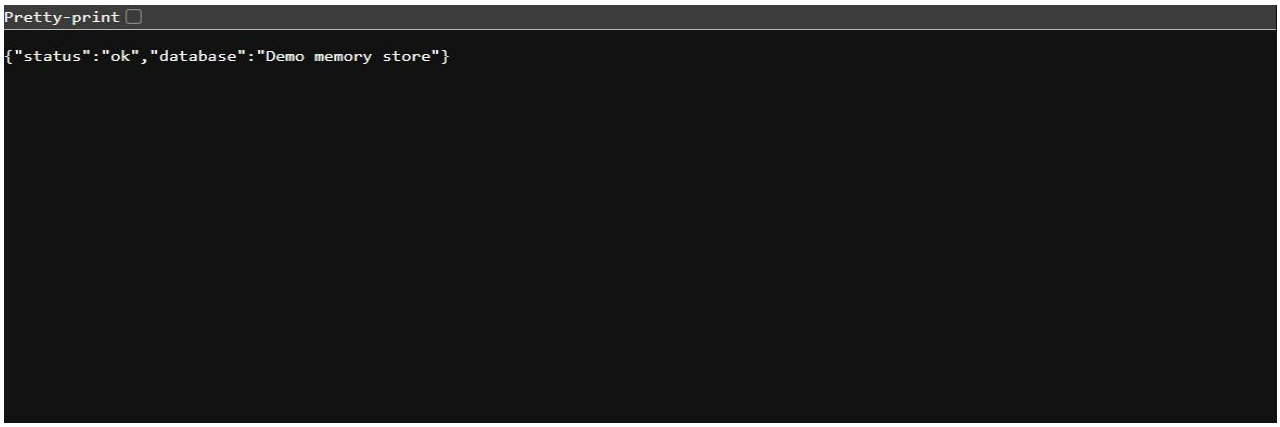


CampusPulse validation error Figure 5.

Duplicate-email protection displaying an API validation error.

Screen 6: Backend API health check

Open http://localhost:5000/api/health and capture the JSON response.



CampusPulse health endpoint

Figure 6. Health endpoint reporting that the API is running with the demo memory store.

Screen 7: Delete confirmation

Click Delete for a test record and capture the browser confirmation dialog before selecting OK. After confirmation, capture the refreshed directory to prove that the record was removed.

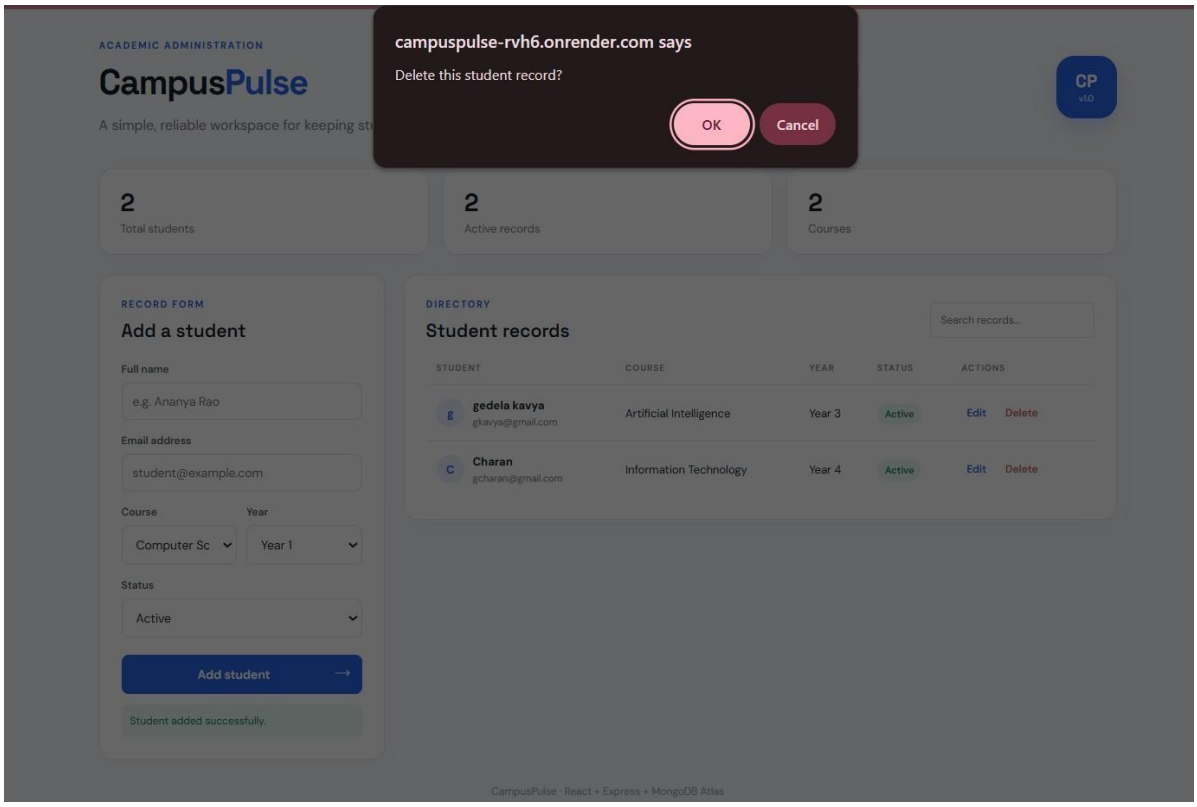


Figure 7. deleting student records from existing student details

The confirmation image is evidence of the prompt; deletion should be verified separately by checking that the record is no longer listed.

Screen 8: Automated test result

Run the following command from the backend folder and capture the terminal output:

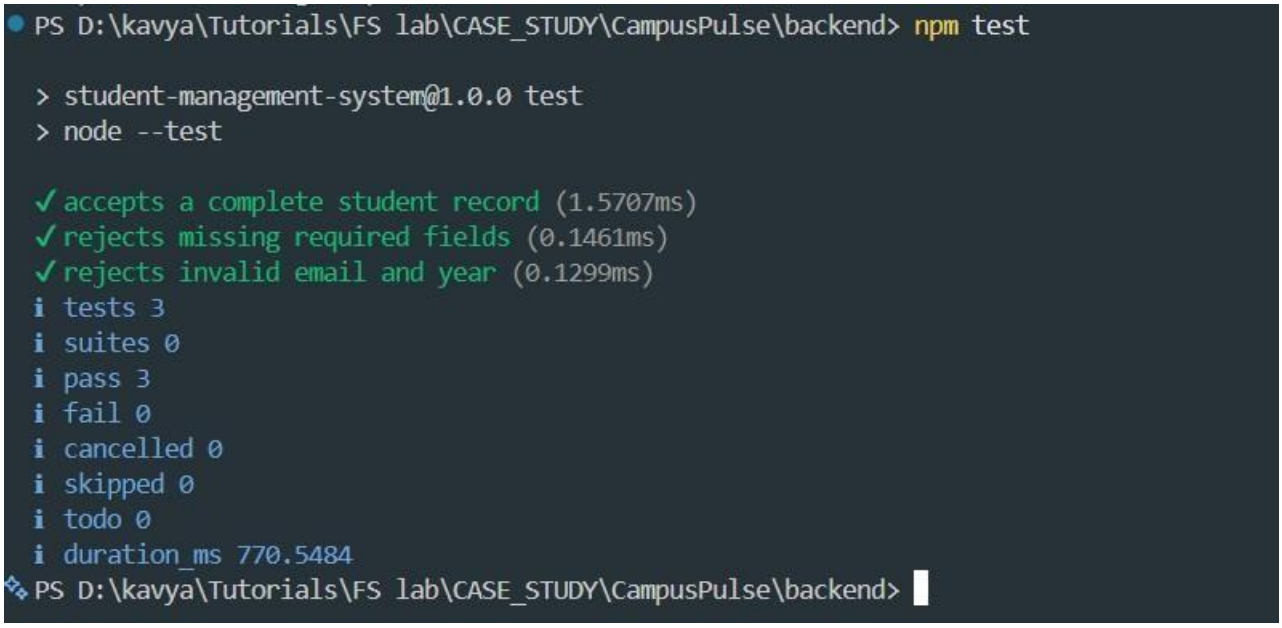


Figure 8. Node.js test runner output showing all CampusPulse validation tests passing.

Screen 9: Database and deployment evidence

If MongoDB Atlas is configured, capture the students collection with a stored document. If deployment is completed, also capture the Render service showing Live status and the Vercel project showing Ready status.

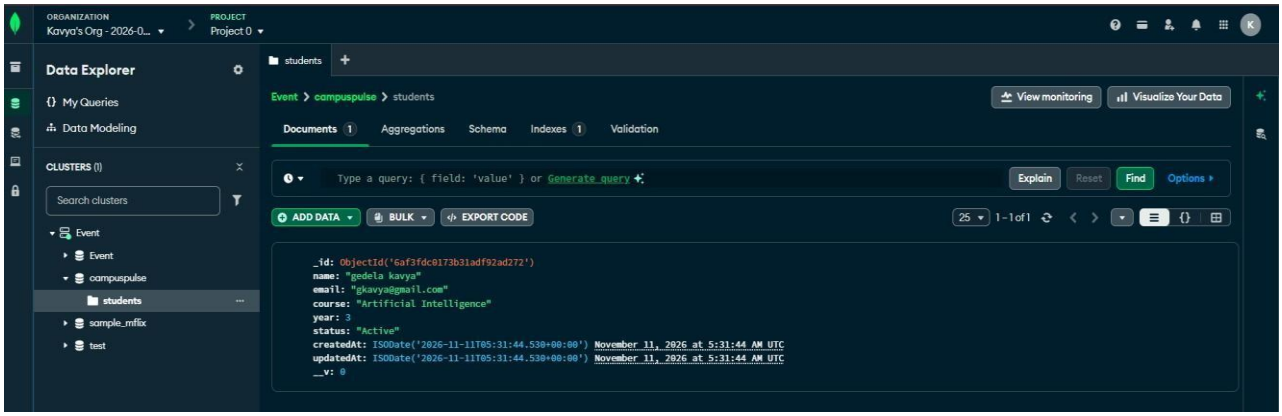
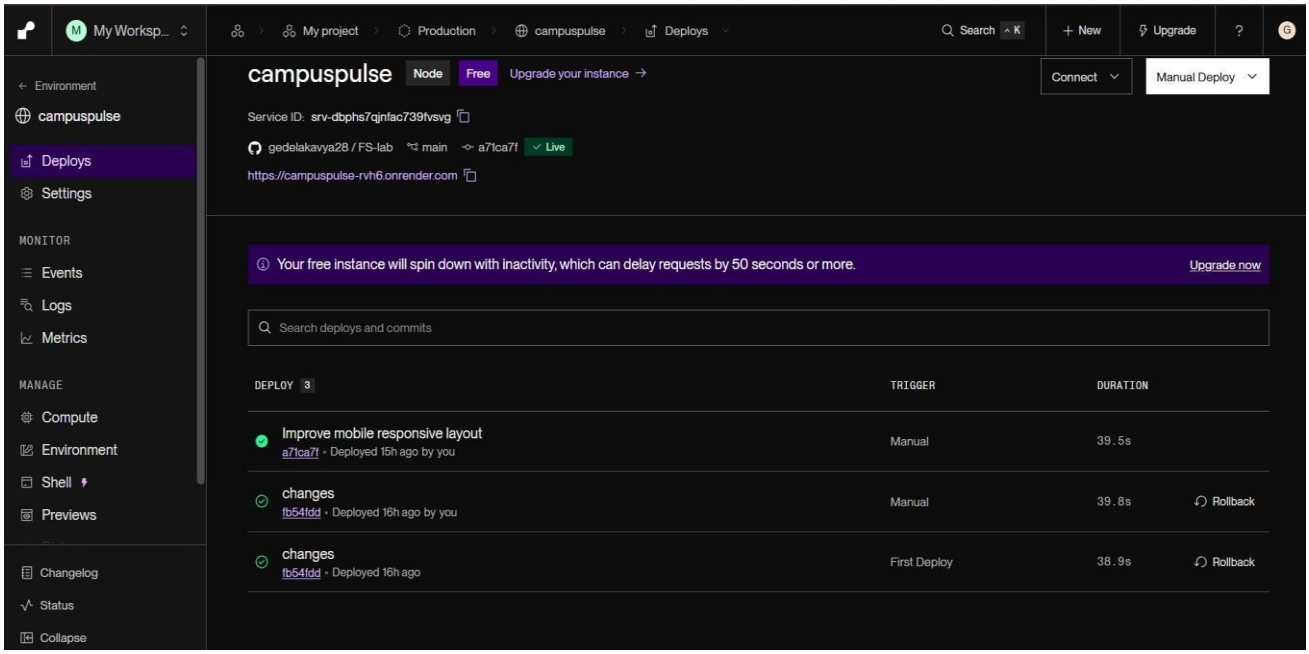


Figure 9. MongoDB Atlas Data Explorer showing a persisted student document in the students collection.



Conclusion

CampusPulse demonstrates how a small full-stack application can replace an error-prone manual studentrecord process. React provides the interactive dashboard, Express provides the REST API, and MongoDB Atlas provides persistent document storage. The application includes CRUD operations, validation, search, duplicate-email protection, health monitoring and a demo memory store for safe local demonstrations.

Future versions can add authentication, attendance, marks, reports and role-based access.